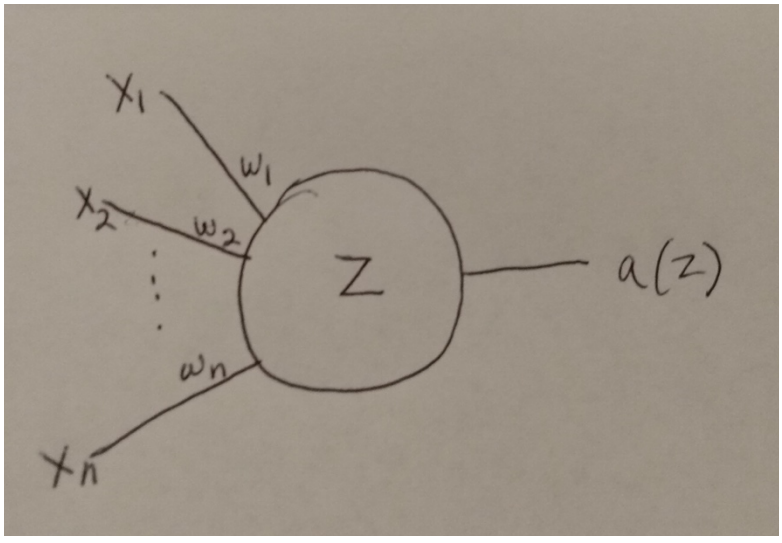
ECE 2036 Lab 3: Neural Networks

Due: Friday March 13, 11:59 PM

In this lab, you will build a simple 2-layer neural network in C++ and train it to recognize hand-written digits. The network will be trained and then tested on the MNIST dataset of digit images. Each image in this dataset is a 28x28 array of pixel values (784 pixels per image). Examples of these images are shown below.

As you can see, some of the digits are very clearly written and others are hard to recognize even for a human. Image recognition is considered a hard task for machines but, as you will discover in the lab, a properly-trained 2-layer neural network can achieve 85-90% accuracy on this task over the MNIST dataset.

As discussed in class, a neural network consists of a series of layers of neurons with connections from one layer to the next. A single neuron performs a very simple computation as shown below:



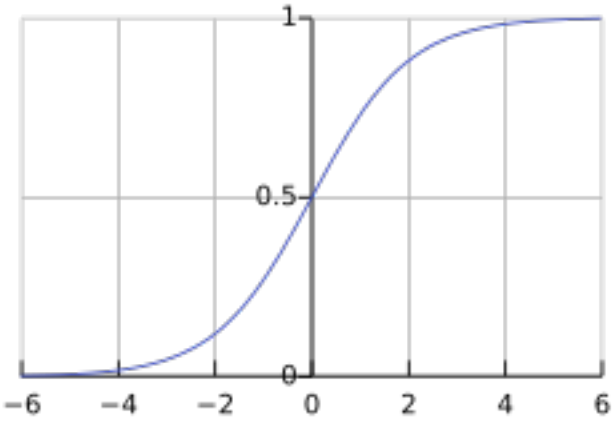
The value z is a weighted sum of the x_i inputs offset by a value called the bias and a(z) is the activation function, which typically applies a threshold to z that determines whether the neuron "fires" or not. A neuron's behavior can be described mathematically as follows:

$z = b + \Sigma\, w_i\, x_i$ or in vector form: $z = WX + b,$

where b is the bias. The most common activation function is the sigmoid function, which is defined as:

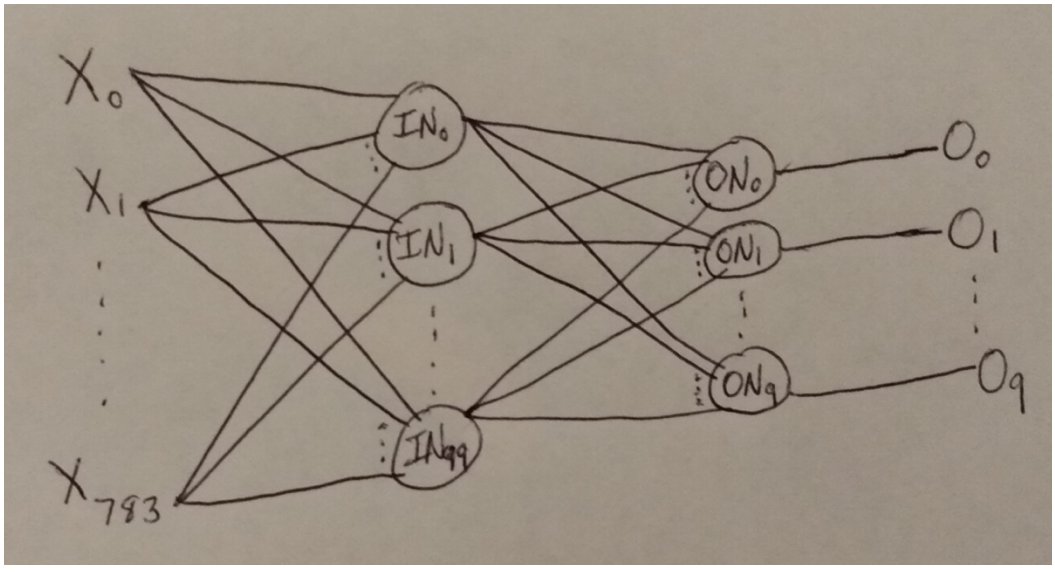
$a(z) = \sigma(z) = 1/(1 + e^{-z})$

A plot of the sigmoid function is shown below:



As you can see, for negative z values, the sigmoid is zero, or close to zero, and as z becomes positive, the sigmoid transitions rapidly to one, which represents the firing of the neuron.

The complete 2-layer neural net you will build, train, and test in the lab looks like:



The input layer has 100 neurons, which can produce 100 independent "features" from the 784 pixels in an image. These features are then fed to the output layer, which classifies the image into 1 of 10 outputs (representing the 10 possible digits) based on the 100 feature values.

The network in this lab is trained using gradient-descent-based backpropagation, which is the most common neural network training method. Gradient descent is used to minimize a loss function, which in this lab is defined as the squared error between the network-predicted outputs for an image and the actual outputs for the image, which are 1 for the correct image label and 0 for all other labels. (The network will produce values between 0 and 1 for each label based on the sigmoid functions at the output layer. These can be loosely thought of as probabilities that the network calculates on each label for an image and the label with the highest probability becomes the predicted output.). For gradient descent, derivatives must be calculated for each weight and each bias in the network in order to move the loss function downward. This begins with the derivative of the loss with respect to the network output, dL/do. The chain rule is then applied in order to find derivatives of weights and biases, first for the output layer and then for the input layer. Since we have defined the loss function for one neuron in the output layer as:

$$L = (o - t)^2$$

where o is the probability value calculated by the network for that output and t is the actual output (referred to as the target value, 0 or 1 depending on whether target is the correct label or not), dL/do is given by:

$$dL/do = 2o - 2t$$

This is the first gradient, which is calculated in MeanSquaredErrorLoss.cc in the code. This is then propagated through the backward() member functions of the network layers to calculate gradients on all weights and biases. As the chain rule derivatives get quite complicated, particularly for the input layer, these backward() member functions are provided for you in the lab.

I have provided a good portion of all the code to implement this neural network. The skeleton code and the MNIST dataset images are available in the course's shared directory on ICE, which is `/storage/ice-shared/ece2036a`. To retrieve the files, type the following Linux commands while in your home directory:

```
cp  /storage/ice-shared/ece2036a/Lab3.tar.gz  .
tar  xvfz  Lab3.tar.gz
```

After executing these commands, you should see a `Lab3` directory containing the source and header files with the shell code in them in your home directory. There should also be a `data` directory, within the `Lab3` directory, containing the image and label files for both training and testing the network.

Once you have completed the code and it compiles correctly, run it as many times as necessary with 2 epochs and 10 test images. You should see the training accuracy increase substantially

from Epoch 1 to Epoch 2 and exceed 80% after the second epoch. ***Warning: Training a neural net is a computation-intensive task and it will likely take 3-5 minutes to run through a single training epoch. Once the network is trained, running over the test image set is relatively fast since it only involves forward passes over the network.*** Output from the random test images should help confirm your code is working correctly. Once you are confident the code is working, run it a few times with 5 epochs. After the second epoch, you should see the accuracy level off or increase slightly over the remaining epochs. Target accuracy is 85% or better after 5 epochs. Note that due to the local optimum issue with gradient descent, there could be somewhat high variability in the accuracy from run to run. However, after several tries, you should be able to achieve the target goal if your code is correct.

Code You Need to Add

There are comments starting with "ADD CODE: …" everywhere you need to write some code to get the neural network working. You will need to write most of the code to implement the forward passes in the network since it is fairly straightforward, mostly just implementing the weighted sum and sigmoid operations at each neuron and the connections between neural layers. For the gradient-descent-based backpropagation, you are only being asked to implement the high-level operations and most of the detailed implementation is provided for you, since the gradient computations are somewhat complicated. Below is a detailed list of all the code you need to add.

digitClassifer.cc:

- in function load_data(): add code to read training labels, training images, test labels, and test images from the files in the data directory
- function accuracy(): this function is empty and must be filled in with code to compute the prediction accuracy for the neural net over a series of predictions
- in main(), add code to do the following:
    - input the number of training epochs (number of iterations of gradient descent over training images that are performed to train the network) and the number of tests to output after the network is trained
    - retrieve the next training image and training label from the dataset
    - make a forward pass over the training image (call the forward() member function for the input layer and then the forward() member function for the output layer with the 100 features produced by the input layer as the inputs for the output layer)
    - set the int "prediction" to the label with the highest output value after the forward pass
    - fill the vector "label_vector" with 1.0 for the correct label and 0.0 for every other label
    - call the forward() member function for the MeanSquaredErrrorLoss object to compute the mean squared error of the outputs from the forward pass relative to the correct values in "label_vector" and add that loss to the double "total_loss" so that mean loss can be reported periodically as the training is going on
    - do the high-level backpropagation steps:
        - calculate the loss gradient by calling the backward() member function on the MeanSquaredErrorLoss object
        - call the backward() member function on the output layer

        - call the backward() member function on the input layer (now the gradients on the weights and biases have all been calculated)
        - call the descend() member function on both layers to apply the gradients
    - repeat above steps for every training image and over every epoch to complete training
    - make forward passes over all the images in the test data set, make a prediction for each image, and report the prediction accuracy over the entire set of test images (this is very similar to what is done during training but skip the backpropagation steps since the model is now fully trained, and also there is only one pass over the images)
    - make forward passes over "numTests" randomly selected images from the test set and compute predictions for each of them (just like the forward passes during training and in computing accuracy over entire test set) – the skeleton code will then print out each of the randomly selected images, the prediction, the correct label and the probabilities the network calculated for each output label, this should give you feedback as to whether your code is working correctly

Neuron.cc

- function forward(): this function is empty and must be filled in by you with weighted sum + bias followed by sigmoid operation
- function descend(): this function is empty and must be filled in by you – note the gradients have already been calculated, here you just apply the gradients to the bias and weights of a neuron; also, "learning_rate" controls the speed of descent and is a multiplicative factor on top of each gradient

NeuralLayer.cc

- function forward(): this function is empty and must be filled in by you, mainly this is just calling the forward function for each neuron in this layer, also must record the current inputs and newly calculated outputs in the vectors "last_input" and "last_output" for use by other parts of the code
- function descend(): this function is empty and must be filled in by you, just call descend() for every neuron in the layer

MeanSquaredErrorLoss.cc

- function forward(): this function is empty and must be filled in by you, compute the squared error for each output (difference between output and target, squared), sum up the squared errors, and return the final sum
- function backward(): for each output, compute dL/do, as specified earlier in the lab write-up, and store the computed gradients in the vector "grad"

Compilation Instructions

While in your `Lab3` directory, type the following commands to compile and run your code:

```
g++ Neuron.cc NeuralLayer.cc MeanSquaredErrorLoss.cc digitClassifier.cc -o run_dc
./run_dc
```

Be sure to compile and test your code this way from a Linux terminal before turning it in!

Sample Output

Here is a sample output from running the program with 3 training epochs and 1 random test image:

```
Seed: 1771960268
Number of labels in training set is 60000
Number of images in training set is 60000
Number of labels in test set is 10000
Number of images in test set is 10000
Data loaded
Train Images: 60000
Train Labels: 60000
Test Images: 10000
Test Labels: 10000

Enter the number of epochs for training: 3

Enter the number of test images to run and output after training: 1

Epoch: 0 | Loss: 0.3943 | Train Accuracy: 0.7324
Epoch: 1 | Loss: 0.2248 | Train Accuracy: 0.8713
Epoch: 2 | Loss: 0.2068 | Train Accuracy: 0.8818
Test Accuracy: 0.8886
values.size(): 784



          .$#
        ., :$O
       :O,       .
              o$
    ,$:        ,$$X
    .O$:       .#$#.
      X$#.     X$$O
      .X@#.  :#$@O
       .o$#:#$$O.
         #$$@$X
        .$$$#,
        ,#$$$o
       ,$$X XX
       :oO  :$:
       .:o  ,$O
      .X$O  o@o
     .:XX .X$X
     ,#. .,$:
      .,,XoXo
      ::  O:.
```

Prediction: 8 | Label: 8
Probabilities: [6.651e-06, 0.002386, 0.0004745, 0.0004305, 0.004033, 0.002791, 0.02079, 0.006786, 0.9641, 0.005407]

Turn-in Instructions

You will compress and bundle your entire directory on ICE and turn this in on Canvas.

1. Make sure the following files and folders are all in your Lab3 directory:

   - MeanSquaredErrorLoss.h (provided with assignment)
   - MeanSquaredErrorLoss.cc (with your code added)
   - NeuralLayer.h (provided with assignment)
   - NeuralLayer.cc (with your code added)
   - Neuron.h (provided with assignment)
   - Neuron.cc (with your code added)
   - digitClassifier.cc (with your code added)
   - data folder (with training and test images, provided with assignment)

2. From your home directory, type the following command:

   ```
   tar cvzf <yourusername>Lab3.tar.gz ./Lab3
   ```

3. Submit the file <yourusername>Lab3.tar.gz via Canvas. (As detailed in Lab 1, it is ***your*** responsibility to make sure that your tarball contains the proper files.)

APPENDIX A: ECE 2036 Lab Grading Rubric

If a student's program runs correctly and produces the desired output, the student has the potential to get a 100 on his or her lab; however, TA's will **randomly** look through this set of "perfect-output" programs to look for other elements of meeting the lab requirements. The table below shows typical deductions that could occur.

In addition, if a student's code does not compile on `coc-ice.pace.gatech.edu`, then they will have an automatic 30% deduction on the lab. Code that compiles but does not match the sample output can incur a deduction from 10% to 30% depending on how poorly the output matches the output specified by the lab. This is in addition to the other deductions listed below or due to the student not attempting the entire assignment.

AUTOMATIC GRADING POINT DEDUCTIONS

Element	Percentage Deduction	Details
Does Not Compile on ICE Cluster	30%	Program does not compile on ICE cluster!
Output Incorrect	10%-30%	Your output should approximately match the sample output

ADDITIONAL GRADING POINT DEDUCTIONS

Element	Percentage Deduction	Details
Is not turned in with a tarball	10%	Just to be clear on the submission
Code correctness	70%	The TA will inspect your code to make this determination.
Clear Self-Documenting Coding Styles	5%-15%	This can include no indentation, using unclear variable names, unclear comments, or compiling with warnings.

LATE POLICY

Element	Percentage Deduction	Details
Late Deduction	See details	< 2 hours late: 5 points off total > 2 hours late, up to 1 day late: 10 points off total > 1 day late, up to 2 days late: 20 points off total 20 additional points off for each day late beyond 2 days

APPENDIX B: C++ Vectors

Vectors are a common data structure in many C++ programs and are extensively used in the neural network code. Vectors in C++ are templated types so that you can create a vector of elements of any type by specifying the type when you instantiate the vector, e.g. the following statement creates a vector of integers named vec:

```
vector<int> vec;
```

Once created, you can add elements to the end of a vector using its push_back() member function, e.g. the following code adds the integers 1 to 10 to vector vec:

```
for (int i=1; i <= 10; i++)
{
        vec.push_back(i);
}
```

A nice feature of vectors is that you can access the elements in the vector as if it was an array, i.e. vec[i] refers to the i^{th} element of vector vec. As an example, the following code outputs the 10 elements of vector vec (note the same indexing as for an array, starting with 0):

```
for (int i=0; i <= 9; i++)
{
        cout << vec[i] << " ";
}
cout << endl;
```

Note that vectors of doubles are used in lots of places in the neural network code and it defines a short cut to make it easy to instantiate new vectors of doubles. The following line in digitClassifier.cc created an alias for a vector of doubles:

```
#define VD vector<double>
```

Note that each image from the training or test set is a vector of doubles, i.e. you can use the following statement to create a vector to hold a single image:

```
VD image;
```

Although it might seem odd, the entire set of training images (also test images) is a vector of images, so it is a vector of vectors of doubles, hence the definitions near the beginning of main():

```
vector<VD> images_train;

vector<VD> images_test;
```